

LAB # 6**A* SEARCH FOR GRAPHS****Objectives:**

- To implement optimal A* algorithm for graphs using python

Hardware/Software Required:

Hardware: Desktop/ Notebook Computer

Software Tool: Python 2.7/ 3.6.2

Introduction:

A* is the best form of greedy first search that gives path with optimal cost for reaching goal from the start node. A* search not only checks the heuristic value at each node, but it also adds the path cost for reaching that node as well along with the heuristics through which the global optimal path is achieved. The shortest path is obtained through Dijkstra's algorithm where the path cost at each node is calculated as:

$$f(n) = g(n) + h(n)$$

Where $f(n)$ is the path cost at n^{th} node, $h(n)$ is the heuristic value of n^{th} node to goal, $g(n)$ is the cost for reaching n^{th} node from start. The pseudo code for the A* search is given below

- let openList equal empty list of nodes
- let closedList equal empty list of nodes
- put startNode on the openList (leave it's f at zero)
- Create a dictionary named costSoFar. Initilize it for startNode
- Create a dictionary named cameFrom. Initilize it for startNode
- while openList is not empty
- for node in openList
- let currentNode equal the node with the least f value
- remove currentNode from the openList
- add currentNode to the closedList
- if currentNode is the goal, You've found the exit!
- for each child in the neighbors
- if child is in the closedList
- continue to beginning of for loop
- $\text{child.g} = \text{currentNode.g} + \text{distance b/w child and current}$
- $\text{child.h} = \text{distance from child to end}$
- $\text{child.f} = \text{child.g} + \text{child.h}$

- if child.position is in the openList's nodes positions
- if child.g is higher than the openList node's g continue to beginning of for loop
- add the child to the openList
- when current=goal
- initialize path list
- append current in path
- update current to cameFrom[current]
- append start to path
- reverse path
- return path

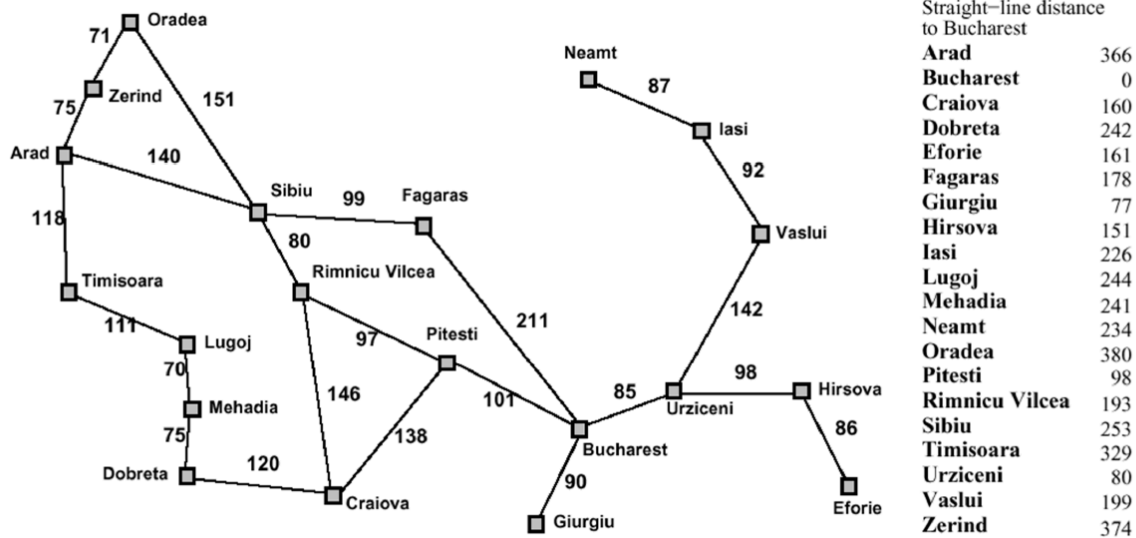
CAMEFROM: USED TO STORE PATH FOLLOWED TO ACCESS A CERTAIN NODE

COSTSOFAR: THIS DICTIONARY WILL TELL US THE VALUE OF TOTAL COSTS FROM GOAL TO THE CURRENT NODE

Lab Tasks:

1. Implement Priority Queue.
2. Implement A* algorithm in python for following graph:

Graph1: Start Node: Arad, Goal: Bucharest



Graph 1

3. Write a script to decompose the given image into an undirected graph where the pixel represents the vertices and adjacent vertices are connected to each other via 4-connectivity and the cost on edges between adjacent nodes is their absolute intensity differences. The heuristic values should be calculated by taking the Manhattan distance for each pixel coordinates. The Manhattan distance between two points (x_0, y_0) and (x_1, y_1) can be calculated as:

$$D = |y_1 - y_0| + |x_1 - x_0|$$

Use A* algorithm to traverse decomposed image starting from pixel 150 to pixel 165.

150	2	5
80	145	45
74	102	165

Conclusion:

Write the conclusion about this lab

NOTE: A lab journal is expected to be submitted for this lab.